claude-windows / dbc4cdae-04f6-4d06-af90-397bbce3fc57

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\subagents\agent-a5097ecf57a5fda88.jsonl`
- SHA-256: `dc69a29123d9848cf6336651249488c1d49b55e1f6f589295d9c265f51dc405d`
- Source modified: `2026-07-06T19:01:05+00:00`
- Imported at: `2026-07-08T15:59:36+00:00`
- project: `subagents`
- session_id: `dbc4cdae-04f6-4d06-af90-397bbce3fc57`

Transcript

1. user (2026-07-06T18:56:44.044Z)

Research spike for issue #349 (badminton-highlight-indexer). Goal: recommend which GPU hardware-decode (NVDEC) library is viable to feed the WASB shuttle-detector on an NVIDIA L4, given the PINNED environment: conda env "wasb" with **torch 1.11.0+cu113, torchvision 0.12.0+cu113, python 3.8** (see deploy/cloudrun/Dockerfile and deploy/digitalocean/gpu_setup.sh in the repo at C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer). Do NOT change code.

First READ the exact loader interface the decode path must satisfy: backend/pipeline/detectors/wasb_infer.py ? functions `make_video_frame_loader` (~L740-780, uses cv2.VideoCapture + cap.set(CAP_PROP_POS_FRAMES)), the frame?tensor conversion (~L320-340), `SequentialFrameStore` (~L280-300), and how frames are batched (DataLoader / _PrefetchIter, ~L590-640). Note: WASB reads frames SEQUENTIALLY in sliding windows (stride 1); frames are resized to a fixed network input (cfg model inp_width/inp_height); random seek is used only for resume.

Then evaluate these candidate GPU-decode approaches and RANK them for THIS pinned env:
1. OpenCV built with CUDA ? cv2.cudacodec.VideoReader
2. ffmpeg -hwaccel cuda / h264_cuvid (via PyAV, or subprocess piping raw frames)
3. NVIDIA DALI (fn.decoders.video / video reader)
4. PyNvVideoCodec (NVIDIA's python NVDEC bindings)
5. torchvision.io GPU VideoReader (device="cuda")
6. decord with GPU

For EACH: (a) compatibility with torch 1.11.0+cu113 / py3.8 / CUDA 11.3 (and whether it needs a newer CUDA/driver ? the serving box is CUDA 12.x driver 580 on L4, but the conda env is cu113); (b) install friction in a conda/micromamba env; (c) whether decoded frames stay ON the GPU (avoid CPU round-trip) or come back to host; (d) API fit for sequential read + GPU resize to fixed WxH + hand-off as a CUDA tensor; (e) known gotchas / maintenance status; (f) how it maps onto the existing loader interface.

Use web search for current compatibility facts (version matrices, known issues) and cite sources.

DELIVERABLE (concise): a ranked recommendation (primary + fallback) with rationale, a compatibility table, the concrete integration sketch against wasb_infer.py's loader (what to add behind a `decode_backend: "cpu"|"nvdec"` flag, default cpu), and the top risks / open questions (esp. the cu113-vs-driver and color/chroma-parity concerns). This informs a default-OFF, parity-gated implementation ? flag anything that would force an env bump.

2. assistant (2026-07-06T18:56:50.925Z)

I'll start by pulling the target repo fresh (per convention), then read the exact loader interface in wasb_infer.py before evaluating the decode backends.

3. user (2026-07-06T18:56:55.720Z)

```
/c/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
true
Already up to date.
```

4. user (2026-07-06T18:56:56.518Z)

```
total 100
drwxr-xr-x 1 avidu 197609    0 Jul  5 22:11 .
drwxr-xr-x 1 avidu 197609    0 Jul  6 16:06 ..
drwxr-xr-x 1 avidu 197609    0 Jun 23 09:53 Annotation Setup
-rw-r--r-- 1 avidu 197609  905 Jul  2 21:20 AUDIT_REPORT_khelsutra-guru_2026-07-02.md
drwxr-xr-x 1 avidu 197609    0 Jul  6 23:47 badminton-highlight-indexer
-rw-r--r-- 1 avidu 197609 1222 Jul  2 21:35 bash.exe.stackdump
drwxr-xr-x 1 avidu 197609    0 Jul  5 12:57 khelsutra
drwxr-xr-x 1 avidu 197609    0 Jul  5 18:47 khelsutra-screencast
drwxr-xr-x 1 avidu 197609    0 Jul  5 13:44 marketing
drwxr-xr-x 1 avidu 197609    0 Jul  5 13:43 pr-158-screenshots
drwxr-xr-x 1 avidu 197609    0 Jun 22 15:03 rally-annotator
drwxr-xr-x 1 avidu 197609    0 Jul  4 06:58 sports-data-collector
drwxr-xr-x 1 avidu 197609    0 Jul  2 19:46 sports-obsreport
---SIBLING---
total 13
drwxr-xr-x 1 avidu 197609   0 Jul  2 11:41 .
drwxr-xr-x 1 avidu 197609   0 Jul  6 16:06 ..
drwxr-xr-x 1 avidu 197609   0 Jul  2 22:08 .claude
-rw-r--r-- 1 avidu 197609 600 Jul  2 11:41 AGENTS.md
drwxr-xr-x 1 avidu 197609   0 Jun 30 03:00 output
```

5. assistant (2026-07-06T18:57:01.249Z)

The repo is confirmed at `C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer` and is up to date. Now let me read the exact loader interface in `wasb_infer.py` and the pinned-env files in parallel.

6. user (2026-07-06T18:57:02.786Z)

```
270         """In-memory sliding buffer over a forward-only frame reader, sized for the
271         detector's sliding-window access pattern (peak O(window) frames, not O(video)).
272
273         The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
274         ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
275         in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
276         start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
277         boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
278         far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
279         earliest index any not-yet-finished window can still ask for) and decode each source
280         frame exactly once via the forward-only ``reader(index)``.
281         """
282
283         def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
284             self._reader = reader
285             self._total = int(total)
286             self._window = max(1, int(window))
287             self._buf: dict = {}
288             # On a resumed run the detector's first needed frame is `start_index`; begin
289             # pulling there so the reader can skip (seek past) the already-cached prefix
290             # instead of decoding 0..start_index-1 just to throw them away.
291             self._next = int(start_index)  # next index not yet pulled from the reader
292             self._max_seen = int(start_index) - 1
293             self._floor = int(start_index)  # lowest index we still keep (never evict below)
294
295         def get(self, index: int):
296             index = int(index)
297             if index < self._floor:
```

```
                raise KeyError(
                    f"frame {index} already evicted (below floor {self._floor}); the access
"
                    f"pattern must stay within a window of the highest frame seen"
                )
            if index >= self._total:
                raise KeyError(f"frame {index} out of range (total {self._total})")
            # Pull forward from the reader until the requested index is buffered.
            while self._next <= index:
                self._buf[self._next] = self._reader(self._next)
                self._next += 1
            if index > self._max_seen:
                self._max_seen = index
            # Evict frames older than the earliest a still-running window could re-request:
            # once we've seen index `m`, no future window starts before `m - (window-1)`.
            new_floor = max(self._floor, self._max_seen - (self._window - 1))
            for k in range(self._floor, new_floor):
                self._buf.pop(k, None)
            self._floor = new_floor
            return self._buf[index]


    def _index_from_synthetic_path(path: str) -> int:
        """Invert ``synthetic_frame_path``: a frame path -> its integer source index.

        Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
        in lockstep with the on-disk naming scheme.
        """
        return _frame_id(path)


    def _bgr_to_pil_rgb(frame_bgr):
        """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
EXACT
        PIL RGB image WASB's disk path produces.

        The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
Image.open(PNG).convert('RGB')``;
        because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
COLOR_BGR2RGB)``
        bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
streamed
        frame indistinguishable from the on-disk one to everything downstream.
        """
        import cv2
        from PIL import Image

        return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))


    def _make_streamed_read_image(store, real_read_image):
        """Build the ``read_image`` replacement used during a streaming detector pass.

        A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
image,
        matching ``read_image``'s real return type); any path that doesn't parse to an index
falls
        through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
testable
        without the WSL-only ``dataloaders`` package.
        """

        def _read_image(path, *args, **kwargs):
            idx = _index_from_synthetic_path(path)
            if idx < 0:
```

```
355                 return real_read_image(path, *args, **kwargs)
356             return _bgr_to_pil_rgb(store.get(idx))
357
358         return _read_image
359
360
361     @contextmanager
362     def _streaming_read_image_patch(
363         frame_loader, total: int, window: int = 1, start_index: int = 0
364     ):
365         """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
decoded
366         frames during the detector pass, byte-for-byte as it would over real PNGs.
367
368         WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
369         ``cv2.imread``. ``read_image`` (``utils.utils``) does
``Image.open(path).convert('RGB')``,
```

7. user (2026-07-06T18:57:03.765Z)

```
580         if remaining:
581             _, transform_test = build_img_transforms(cfg)
582             try:
583                 _, seq_transform_test = build_seq_transforms(cfg)
584             except Exception:
585                 seq_transform_test = None
586             ds = ImageDataset(
587                 cfg,
588                 remaining,
589                 input_wh,
590                 output_wh,
591                 transform=transform_test,
592                 seq_transform=seq_transform_test,
593                 is_train=False,
594             )
595             # Streaming forces single-process loading: the in-memory frame store + the
596             # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
597             loader_workers = 0 if streaming else num_workers
598             loader = DataLoader(
599                 ds, batch_size=batch_size, shuffle=False, num_workers=loader_workers
600             )
601             detector = build_detector(cfg)
602
603             from contextlib import ExitStack
604
605             t0 = time.time()
606             done = windows_done
607             win_idx = windows_done
608             log_every = max(1, log_every_batches)
609             flush_n = max(1, flush_every)
610             pending = []
611             logger.info(
612                 f"detecting on {len(remaining)} remaining windows "
613                 f"(total {windows_total}, batch={batch_size}, workers={loader_workers}, "
614                 f"source={'video-stream' if streaming else 'frames-dir'})..."
615             )
616             det_f = open(det_jsonl, "a")
617             try:
618                 with ExitStack() as stack:
619                     stack.enter_context(torch.no_grad())
620                     # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
to the
621                     # in-memory decoded frame for that synthetic path; every other line of
622                     # __getitem__ runs unchanged so the tensor matches the PNG round-trip
```

```
exactly.
623                     # The detector's first needed frame is `windows_done` (window i starts
at
624                     # frame i), so prime the store there for a resume.
625                     if streaming:
626                         stack.enter_context(
627                             _streaming_read_image_patch(
628                                 frame_loader,
629                                 len(frames),
630                                 window=frames_in,
631                                 start_index=windows_done,
632                             )
633                         )
634                     # Overlap CPU batch assembly with GPU inference (streaming/workers=0
only ?
635                     # a multi-worker DataLoader already overlaps via its worker processes).
636                     batches = (
637                         _PrefetchIter(loader, depth=prefetch_batches)
638                         if prefetch_batches and loader_workers == 0
639                         else loader
640                     )
641                     for bi, batch in enumerate(batches):
642                         imgs, _hms, trans = batch[0], batch[1], batch[2]
643                         img_paths = [
644                             list(t) for t in batch[-1]
645                         ]  # [frame_in][batch] -> path
646                         batch_results, _ = detector.run_tensor(imgs, trans)
647                         nb = imgs.shape[0]
648                         for ib in range(nb):
649                             frames_payload = []
650                             for ie in sorted(batch_results[ib].keys()):
651                                 p = img_paths[ie][ib]
652                                 fid = _frame_id(p)
653                                 plain = [_det_plain(d) for d in batch_results[ib][ie]]
654                                 frames_payload.append([fid, plain])
655                                 det_by_fid[fid].extend(plain)
656                             pending.append({"w": win_idx, "f": frames_payload})
657                             win_idx += 1
658                         done += nb
659                         if len(pending) >= flush_n or done >= windows_total:
```

8. user (2026-07-06T18:57:04.574Z)

```
730
731     def _probe_frame_count(video_path: str) -> int:
732         """Container-reported frame count via cv2 (0 if unknown). Used to size the
stream."""
733         import cv2
734
735         cap = cv2.VideoCapture(video_path)
736         if not cap.isOpened():
737             raise SystemExit(f"cannot open video: {video_path}")
738         n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
739         cap.release()
740         return n
741
742
743     def make_video_frame_loader(video_path: str):
744         """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.
745
746         ``frame_loader(index) -> BGR uint8 ndarray`` decodes the video forward-only with one
747         long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
748         detector's sliding-window order); it reads sequentially and only uses a frame-seek
to
749         skip forward when a *resume* starts past the current position. Each ``cap.read()``
```

```
        returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
would
        yield (PNG is lossless), so the detector sees identical pixels to the disk path.

        Returns the count hint from the container so the caller can build the frame list;
        ``release`` closes the capture.
        """
        import cv2

        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            raise SystemExit(f"cannot open video: {video_path}")
        count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
        state = {"pos": 0}  # next index cap.read() will return

        def frame_loader(index: int):
            index = int(index)
            if index < state["pos"]:
                raise RuntimeError(
                    f"streaming decode is forward-only; got index {index} after
{state['pos']}"
                )
            if index > state["pos"]:
                # Forward skip only happens when resuming past cached windows. Seek once.
                cap.set(cv2.CAP_PROP_POS_FRAMES, index)
                state["pos"] = index
            ok, frame = cap.read()
            if not ok:
                raise RuntimeError(f"failed to decode frame {index} from {video_path}")
            state["pos"] += 1
            return frame

        def release():
            cap.release()

        return frame_loader, count_hint, release


    def run_video_streaming(
        video_path: str,
        weights: str,
        out_csv: str,
        sport: str = "badminton",
        limit: int = 0,
        batch_size: int = 8,
        log_every_batches: int = 50,
        cache_dir: "str | None" = None,
        flush_every: int = 1000,
        fresh: bool = False,
        device: str = "cuda",
        prefetch_batches: int = 0,
    ) -> str:
        """Fast path: decode ``video_path`` on the fly and feed frames straight to the
        detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
        dominates a long clip). Output is bit-identical to the frames-on-disk path; see
        ``run``'s docstring for why.
        """
        frame_loader, count_hint, release = make_video_frame_loader(video_path)
        try:
            if count_hint <= 0:
                raise SystemExit(
                    f"could not determine frame count for {video_path}; cannot stream "
                    f"(fall back to frame extraction with --frames_dir)"
                )
            frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
```

```
813             source_key = "video:" + osp.abspath(video_path)
814             return run(
815                 frame_list,
816                 weights,
817                 out_csv,
818                 sport,
819                 limit,
820                 batch_size=batch_size,
821                 num_workers=0,
822                 log_every_batches=log_every_batches,
823                 cache_dir=cache_dir,
824                 flush_every=flush_every,
825                 fresh=fresh,
826                 frame_loader=frame_loader,
827                 source_key=source_key,
828                 device=device,
829                 prefetch_batches=prefetch_batches,
```

9. assistant (2026-07-06T18:57:09.009Z)

I have the core loader interface. Now let me read the frame?tensor conversion path, the ImageDataset `__getitem__`, and how `run_tensor` consumes the batch, plus the pinned-env files. Let me look at the transforms and the tensor-building path more closely.

10. user (2026-07-06T18:57:11.516Z)

```
369         ``cv2.imread``. ``read_image`` (``utils.utils``) does
``Image.open(path).convert('RGB')``,
370         returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
feed it a
371         PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
372         byte-identical to the disk round-trip, and the shim never touches the filesystem so
the
373         ``osp.exists`` guard is sidestepped for synthetic stream paths.
374
375         We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
actually
376         calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
in that
377         module's namespace; patching ``utils.read_image`` would NOT rebind the already-
imported
378         reference).
379
380         ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
wrapped
381         in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
on a
382         resume we start at ``start_index`` so the decoder seeks past already-cached windows.
The
383         original reader is restored on exit.
384         """
385         from dataloaders import dataset_loader as _dl
386
387         store = SequentialFrameStore(
388             frame_loader, total, window=window, start_index=start_index
389         )
390         real_read_image = _dl.read_image
391         # Rebind read_image in the consuming module for the duration of the detector pass;
392         # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
B010-clean.)
393         _dl.read_image = _make_streamed_read_image(store, real_read_image)  # type:
ignore[assignment]
394         try:
395             yield store
396         finally:
```

```
            _dl.read_image = real_read_image  # type: ignore[assignment]


    # ---------------------------------------------------------------------- #
    # Inference
    # ---------------------------------------------------------------------- #
    class _PrefetchIter:
        """Bounded producer-thread wrapper over a batch iterator (the num_workers=0
DataLoader).

        In streaming mode the DataLoader must stay single-process (the in-memory frame store
+
        the cv2.imread shim live in this process and cannot be pickled to workers), so the
        CPU-side batch assembly (decode + PIL + ToTensor) SERIALIZES with GPU inference ?
        measured on the first real L4 serving run (2026-07-03) this left the GPU <1%
utilized.
        A single producer THREAD fixes the overlap without multiprocessing: cv2/PIL/numpy
        release the GIL for the heavy parts, the store's forward-only access pattern is
        preserved (one consumer thread iterating sequentially), and the bounded queue caps
        memory at ``depth`` batches. A producer exception is re-raised in the consumer."""

        _END = object()

        def __init__(self, src, depth: int = 2):
            import queue
            import threading

            self._q: "queue.Queue" = queue.Queue(maxsize=max(1, int(depth)))
            self._exc: "BaseException | None" = None

            def _fill() -> None:
                try:
                    for item in src:
                        self._q.put(item)
                except BaseException as e:  # noqa: BLE001 - surfaced in __next__
                    self._exc = e
                finally:
                    self._q.put(self._END)

            self._t = threading.Thread(target=_fill, name="wasb-prefetch", daemon=True)
            self._t.start()

        def __iter__(self):
            return self

        def __next__(self):
            item = self._q.get()
            if item is self._END:
                if self._exc is not None:
                    raise self._exc
                raise StopIteration
            return item


    def run(
        frames,
        weights: str,
        out_csv: str,
        sport: str = "badminton",
        limit: int = 0,
        batch_size: int = 8,
        num_workers: int = 4,
        log_every_batches: int = 50,
        cache_dir: "str | None" = None,
        flush_every: int = 1000,
```

```
459         fresh: bool = False,
460         frame_loader=None,
461         source_key: "str | None" = None,
462         device: str = "cuda",
463         prefetch_batches: int = 0,
464     ) -> str:
465         """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
466
467         Two equivalent ways to supply pixels (output is bit-identical between them):
468
469         * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
470           globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
``cv2.imread``.
471         * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
472           frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
473           pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
474           reader, which returns a PIL RGB image) over the DataLoader pass so every other
line of
475           ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
identical
476           to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
477           ``cvtColor(bgr, BGR2RGB)``, verified byte-identical). Streaming forces
``num_workers=0``
478           (the in-process shim + buffer can't cross worker processes).
479
480         ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
481         frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
482         after reboot still matches.
483         """
484         import time
485
486         import torch
487         from dataloaders import build_img_transforms, build_seq_transforms
488         from dataloaders.dataset_loader import ImageDataset
```

11. user (2026-07-06T18:57:13.504Z)

```
333:    because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
COLOR_BGR2RGB)``
340:    return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
403:class _PrefetchIter:
477:      ``cvtColor(bgr, BGR2RGB)``, verified byte-identical). Streaming forces
``num_workers=0``
487:    from dataloaders import build_img_transforms, build_seq_transforms
505:    input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
506:    output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
581:        _, transform_test = build_img_transforms(cfg)
589:            input_wh,
590:            output_wh,
```

12. assistant (2026-07-06T18:57:18.144Z)

This gives me the full picture of the loader contract. Let me look at the pinned-env files (Dockerfile, gpu_setup.sh) and the `run_tensor` / ImageDataset resize semantics to nail down (a) where resize happens and (b) the exact CUDA/torch pins. Let me find these files.

13. user (2026-07-06T18:57:18.542Z)

```
No files found
```

14. user (2026-07-06T18:57:21.879Z)

```
Found 8 files
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\deploy\cloudrun\README.md
```

C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\deploy\cloudrun\Dockerfile
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\deploy\digitalocean\bootstrap.sh
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\deploy\digitalocean\README.md
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\deploy\gcp\README.md
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\deploy\gcp\nightly_regression\run_on_vm.sh
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\deploy\digitalocean\gpu_setup.sh
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\deploy\cloudrun\cloudbuild.yaml

15. user (2026-07-06T18:57:26.180Z)

```
1       # Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7,
ADR-012/D16).
2       #
3       # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-
located so
4       # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the
owner on the
5       # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too
heavy) ? the
6       # dispatch shim is proven with a fake client (tests/test_compute_backend.py).
7       #
8       # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
9       # native GPU box, deploy/digitalocean/gpu_setup.sh):
10      #   * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
11      #   * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
a subprocess
12      #     (the native runner shells out to $WASB_PYTHON), so the app's torch never co-
installs with it.
13      #
14      # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT
cleared for
15      #   commercial sharing. They are NOT baked into this image; stage them at runtime (a
bucket fetch or a
16      #   mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner
serving.
17
18      FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
19
20      ENV DEBIAN_FRONTEND=noninteractive \
21          PYTHONUNBUFFERED=1 \
22          PIP_NO_CACHE_DIR=1 \
23          CONDA_DIR=/opt/conda \
24          WASB_REPO_DIR=/opt/models/WASB-SBDT
25
26      # ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB
env build.
27      RUN apt-get update && apt-get install -y --no-install-recommends \
28              ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
29              python3 python3-pip python3-venv \
30          && rm -rf /var/lib/apt/lists/*
31
32      # --- WASB detector env (micromamba + conda-forge ? BSD-licensed, NO Anaconda ToS; P3a,
ADR-005) ---
33      # Enforces ADR-005 (every dependency ? code, data, weights, AND the build toolchain ?
stays
34      # MIT/Apache/BSD): micromamba (BSD-3) + conda-forge (community) replaces Miniconda's
Anaconda-ToS-gated
35      # `defaults` channels (the ?200-employee Terms-of-Service trap flagged in
PACKAGING_PLAN.md §3).
36      # MAMBA_ROOT_PREFIX=$CONDA_DIR (=/opt/conda) keeps the env at $CONDA_DIR/envs/wasb so
```

```
$WASB_PYTHON below
37      # is UNCHANGED. Same verified torch-1.11.0+cu113 WASB stack; the pip wheels are
unaffected by the switch.
38      ENV MAMBA_ROOT_PREFIX=/opt/conda
39      RUN curl -sSL https://micro.mamba.pm/api/micromamba/linux-64/latest \
40              | tar -xj -C /usr/local bin/micromamba \
41          && micromamba create -y -q -n wasb -c conda-forge python=3.8 \
42          && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
43          && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
44              torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113 \
45          && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python"
-m pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
46          && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
47              hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib "numpy<1.24"
48
49      # --- Main application env ---
50      WORKDIR /app
51      COPY pyproject.toml README.md ./
52      COPY requirements-trackers.txt ./
53      COPY backend ./backend
54      COPY training ./training
55      # Install with the [storage-gcs,cloud-run,auth] extras. storage-gcs: the cloud worker
PULLS the gs://
56      # input and PUSHES the gs:// outputs/done-marker through backend.storage (google-cloud-
storage is
57      # lazy-imported there); without it the job dies at the first gs:// fetch with
`ModuleNotFoundError:
58      # google.cloud`. cloud-run: this SAME image is reused as the deployed CONTROL PLANE,
which dispatches
59      # the L4 worker Job via google.cloud.run_v2 (backend/compute/cloud_run.py, lazy-
imported); without it
60      # the control plane fails every dispatch with "google-cloud-run is required for
CloudRunCompute's real
61      # client". Both are real Cloud Run failures the mock-client tests can't see (the SDKs
are faked in CI).
62      # auth: PyJWT[crypto] for M9 edge-auth (security.edge_auth_enabled ? the Cf-Access JWT
verify the
63      # reused-as-control-plane deployment runs behind api.khelsutra.guru); jwt is lazy-
imported and
64      # edge-auth is default-OFF, so the worker path is unaffected ? but without it a control-
plane
65      # deploy with edge-auth ON needed a hand-built derived image (the A1 deploy's PyJWT
overlay).
66      # D13/P12: `trackers` (ByteTrack's new home) is a git-only dep (broken PyPI wheel ?
67      # roboflow/trackers#474) so it's NOT pulled by `pip install .`; install it out-of-band
before
68      # the main env, or the person-cue path crashes with ModuleNotFoundError. The immutable
git SHA is
69      # single-sourced in requirements-trackers.txt.
70      RUN python3 -m pip install --upgrade pip \
71          && python3 -m pip install -r requirements-trackers.txt \
72          && python3 -m pip install ".[storage-gcs,cloud-run,auth]"
73
74      # The native runner finds the WASB env + repo by env (no `conda activate` needed);
weights come in at
75      # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the
default.
76      # PYTHONUNBUFFERED: without it the worker's stdout/stderr sit in the block buffer until
process
77      # exit, so Cloud Logging shows a single end-of-run burst instead of live progress
(observed on the
78      # first real M7 runs, 2026-07-03) ? a running job looks stalled and mid-run failures log
nothing.
```

```
79      ENV PYTHONUNBUFFERED=1 \
80          WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
81          WASB_REPO_DIR=/opt/models/WASB-SBDT \
82          WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
83          RALLY_DETECTOR_IMPL=native
84
85      # The dispatcher (CloudRunCompute) supplies the per-execution args:
86      #   infer --video-uri gs://? --out-uri gs://? --done-uri gs://? --segmenter wasb_hybrid
--set ?
87      ENTRYPOINT ["python3", "-m", "backend.worker"]
88
```

16. user (2026-07-06T18:57:26.740Z)

```
1       #!/usr/bin/env bash
2       #
3       # Native (no-WSL) WASB shuttle-detector setup for a Linux GPU box ? the M3.5 enabler for
4       # real trajectory extraction. Replicates docs/TRACKNET_WSL_SETUP.md's WSL conda env
5       # *natively* on Linux. Run AFTER ./bootstrap.sh --gpu (which provides CUDA torch for the
6       # main venv); this builds a SEPARATE conda env for WASB (torch 1.11) ? never co-
installed
7       # with the indexer's torch (framework conflict).
8       #
9       # Requires an NVIDIA GPU + driver (the DO `gpu-*-base` image ships it).
10      #
11      # ? WASB-C3: the pretrained badminton weights' license / source-dataset provenance is
NOT
12      # verified here. Fine for internal extraction + Gen-0 bootstrapping; clear it before
sharing
13      # any derived trajectories or model. See docs/DECOUPLED_COMPUTE/06.
14      set -euo pipefail
15
16      MODELS_DIR="${RALLY_MODELS_DIR:-$HOME/models}"
17      CONDA_DIR="${CONDA_DIR:-$HOME/miniconda3}"
18      WASB_REPO="$MODELS_DIR/WASB-SBDT"
19      mkdir -p "$MODELS_DIR"
20
21      echo "[gpu] [1/5] miniconda (native Linux)"
22      if [ ! -x "$CONDA_DIR/bin/conda" ]; then
23        curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o
/tmp/mc.sh
24        bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh
25      fi
26      # shellcheck disable=SC1091
27      source "$CONDA_DIR/etc/profile.d/conda.sh"
28
29      # Modern conda BLOCKS non-interactive `conda create` until the defaults-channel Terms of
Service
30      # are accepted (CondaToSNonInteractiveError). Accept them so the env build doesn't fail
on a fresh
31      # box. (Observed on a GCP Deep-Learning-VM image, 2026-06-20.)
32      conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/main
>/dev/null 2>&1 || true
33      conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r
>/dev/null 2>&1 || true
34
35      echo "[gpu] [2/5] WASB-SBDT repo"
36      if [ ! -d "$WASB_REPO/.git" ]; then
37        git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO"
38      fi
39
40      echo "[gpu] [3/5] conda env 'wasb' (py3.8 + torch 1.11.0+cu113 ? the verified WASB
stack)"
41      conda env list | grep -q '/envs/wasb$' || conda create -y -q -n wasb python=3.8
42      conda activate wasb
```

```
43      python -m pip install -q torch==1.11.0+cu113 torchvision==0.12.0+cu113 \
44        --extra-index-url https://download.pytorch.org/whl/cu113
45      [ -f "$WASB_REPO/requirements.txt" ] && python -m pip install -q -r
"$WASB_REPO/requirements.txt" || true
46      # WASB-SBDT's requirements.txt is INCOMPLETE: src/ imports hydra, pandas, etc. that it
omits, so
47      # `import wasb_infer` deps fail without these. (Found on the GCP L4 port, 2026-06-20.)
numpy is
48      # pinned <1.24 for torch-1.11 ABI compatibility.
49      python -m pip install -q hydra-core omegaconf pandas opencv-python-headless pillow scipy
tqdm pyyaml matplotlib "numpy<1.24" || true
50
51      echo "[gpu] [4/5] pretrained badminton weights"
52      WEIGHTS="$WASB_REPO/pretrained_weights/wasb_badminton_best.pth.tar"
53      if [ -f "$WEIGHTS" ]; then
54        echo "[gpu]       found $WEIGHTS"
55      else
56        echo "[gpu]       !! MISSING $WEIGHTS ? fetch wasb_badminton_best.pth.tar
(Drive/bucket) before extraction."
57      fi
58
59      echo "[gpu] [5/5] GPU sanity check (in the wasb env)"
60      python - <<'PY'
61      import torch
62      ok = torch.cuda.is_available()
63      print("[gpu]       torch", torch.__version__, "| cuda_available", ok,
64            "| device", (torch.cuda.get_device_name(0) if ok else "CPU"))
65      assert ok, "CUDA not available in the wasb env ? check the GPU driver / base image"
66      PY
67
68      echo "[gpu] native WASB env ready.  Activate with:  source
$CONDA_DIR/etc/profile.d/conda.sh && conda activate wasb"
69      echo "[gpu] (the LinuxNativeRunner ? backend/pipeline/detectors ? drives extraction once
it lands; M3.5.)"
70
```

17. assistant (2026-07-06T18:57:39.485Z)

I now have the exact pinned environment and loader contract. The key architectural facts:

- **wasb env is a subprocess**: `torch 1.11.0+cu113, torchvision 0.12.0+cu113, python 3.8`, opencv-python-**headless** (no CUDA), numpy<1.24. Base image is `nvidia/cuda:12.4.1-runtime`, host driver 580 / CUDA 12.x on L4.
- **Loader contract**: `frame_loader(index) -> BGR uint8 ndarray`, forward-only, seek only on resume. It flows into `SequentialFrameStore` ? `_bgr_to_pil_rgb` (`cvtColor BGR2RGB` ? PIL RGB) ? WASB's `ImageDataset.__getitem__` via the `read_image` shim ? `transform_test` (which does the resize to `inp_width/inp_height`) ? DataLoader ? `detector.run_tensor(imgs, trans)`.
- **Critical parity constraint**: the code goes to great lengths to be **byte-identical** to the PNG-on-disk path. Resize happens *inside* WASB's transforms on a full-res BGR/PIL frame, NOT in the loader.

Let me confirm one detail ? where exactly the resize lives (in `build_img_transforms` / `ImageDataset`) ? then run web searches for the compatibility facts. Let me check `run_tensor` and the transform to confirm resize location, and kick off the web research in parallel.

18. user (2026-07-06T18:57:42.670Z)

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_infer.py:646:                    batch_results, _ =
detector.run_tensor(imgs, trans)
```

19. assistant (2026-07-06T18:57:47.942Z)

The resize/transform lives inside the external WASB repo (`dataloaders`, `ImageDataset`), not

this file ? confirming that resize is done by WASB on the CPU PIL frame, downstream of the loader. That's the key parity boundary. Let me confirm `run_tensor` takes an already-assembled CPU tensor batch (moves to GPU internally) and check the `trans` affine-transform semantics, then launch the web research.

20. user (2026-07-06T18:57:49.454Z)

```
500
501         streaming = frame_loader is not None
502
503         cfg = _build_cfg(weights, sport, device)
504         frames_in = int(cfg["model"]["frames_in"])
505         input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
506         output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
507
508         if streaming:
509             # `frames` is already the ordered list of (synthetic) frame paths.
510             if not isinstance(frames, (list, tuple)):
511                 raise SystemExit(
512                     "streaming mode requires `frames` to be a list of frame paths"
513                 )
514             frames = list(frames)
515             frames_dir = source_key or "stream"
516         else:
517             frames_dir = frames
518             frames = sorted(
519                 glob.glob(osp.join(frames_dir, "*.png"))
520                 + glob.glob(osp.join(frames_dir, "*.jpg"))
521             )
522         if limit:
523             frames = frames[:limit]
524         if len(frames) < frames_in:
525             where = frames_dir if not streaming else "video stream"
526             raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
527
528         # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
529         samples = []
```

21. user (2026-07-06T18:57:50.576Z)

```
Found 5 files
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\wasb_infer.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\native_wasb_runner.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\device.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py
```

22. assistant (2026-07-06T18:57:58.753Z)

I have enough of the loader contract. The `run_tensor(imgs, trans)` receives a CPU tensor batch (assembled by the DataLoader from PIL frames via `transform_test`) plus the affine `trans` matrices for mapping heatmap coords back to source pixels; it moves `imgs` to the GPU internally. The critical parity boundary is clear: **resize + normalize happen inside WASB's CPU transforms on the full-res PIL frame, downstream of the loader**, and the code is engineered to be byte-identical to the PNG-on-disk path.

Now let me run the web research on the six candidate decoders. I'll batch the searches.

23. user (2026-07-06T18:58:08.175Z)

Web search results for query: "NVIDIA DALI fn.decoders.video python 3.8 CUDA 11.3 minimum supported version compatibility matrix"

Links: [{"title":"NVIDIA DALI Documentation ? NVIDIA DALI","url":"https://docs.nvidia.com/deeplearning/dali/user-guide/docs/"},{"title":"Releases · NVIDIA/DALI","url":"https://github.com/NVIDIA/DALI/releases"},{"title":"GitHub - NVIDIA/DALI: A GPU-accelerated library containing highly optimized building blocks and an execution engine for data processing to accelerate deep learning training and inference applications. · GitHub","url":"https://github.com/NVIDIA/DALI"},{"title":"Release Notes ? NVIDIA DALI Release Notes","url":"https://docs.nvidia.com/deeplearning/dali/archives/dali_1_48_0/release-notes/index.html"},{"title":"NVIDIA/DALI v1.49.0 on GitHub","url":"https://newreleases.io/project/github/NVIDIA/DALI/release/v1.49.0"},{"title":"nvidia.dali.fn.experimental.decoders.video ? NVIDIA DALI","url":"https://docs.nvidia.com/deeplearning/dali/user-guide/docs/operations/nvidia.dali.fn.experimental.decoders.video.html"},{"title":"Installation ? NVIDIA DALI","url":"https://docs.nvidia.com/deeplearning/dali/user-guide/docs/installation.html"},{"title":"nvidia.dali.fn.readers.video ? NVIDIA DALI","url":"https://docs.nvidia.com/deeplearning/dali/user-guide/docs/operations/nvidia.dali.fn.readers.video.html"},{"title":"nvidia.dali.fn.decoders.image ? NVIDIA DALI","url":"https://docs.nvidia.com/deeplearning/dali/user-guide/docs/operations/nvidia.dali.fn.decoders.image.html"},{"title":"nvidia.fn.experimental.decoders.video does not return the correct frame for higher resolution videos · Issue #4709 · NVIDIA/DALI","url":"https://github.com/NVIDIA/DALI/issues/4709"}]

Based on the search results, I found relevant information about NVIDIA DALI and Python/CUDA compatibility:

Key Findings:

DALI 1.49 is the last release to support Python 3.8. This means if you're using Python 3.8, you'll need to use DALI version 1.49 or earlier.

Regarding CUDA 11.3 compatibility, CUDA 11.0 and CUDA 12.0 builds use CUDA toolkit enhanced compatibility. However, the search results show that CUDA 11 build is provided starting from DALI 0.22.0, which means CUDA 11.x versions (including CUDA 11.3) are supported across many DALI versions.

Regarding the `fn.decoders.video` operator specifically, the results show it's an experimental feature. experimental.decoders.video does not currently support checkpointing.

For a comprehensive compatibility matrix specific to Python 3.8, CUDA 11.3, and the video decoder operator, you would want to consult the official NVIDIA DALI Installation Guide or the release notes for specific DALI versions you're considering, as the search results don't contain a complete compatibility matrix table for these specific combinations.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

24. user (2026-07-06T18:58:09.202Z)

Web search results for query: "decord GPU nvdec bridge_to_torch maintenance status abandoned 2024 2025 CUDA 11.3"

Links: [{"title":"Accelerated video decoding with NVDEC ? Torchaudio 2.8.0 documentation","url":"https://docs.pytorch.org/audio/main/tutorials/nvdec_tutorial.html"},{"title":"GitHub - dmlc/decord: An efficient video loader for deep learning with smart shuffling that's super easy to digest · GitHub","url":"https://github.com/dmlc/decord"},{"title":"10. Introducing Decord: an efficient video reader ? gluoncv 0.11.0 documentation","url":"https://cv.gluon.ai/build/examples_action_recognition/decord_loader.html"},{"title":"Accelerated video decoding with NVDEC ? Torchaudio 2.1.0 documentation","url":"https://docs.pytorch.org/audio/2.1.0/tutorials/nvdec_tutorial.html"},{"title":"Accelerated Video Decoding with NVDEC ? Torchaudio 0.12.1 documentation","url":"https://docs.pytorch.org/audio/0.12.1/hw_acceleration_tutorial.html"},{"title":"GPU

accelerated video loading with optimizations for reading at specific timestamps or time intervals · Issue #8599 · pytorch/vision","url":"https://github.com/pytorch/vision/issues/8599"},{"title":"NVIDIA NVENC - NamuWiki","url":"https://en.namu.wiki/w/NVIDIA%20NVENC"},{"title":"Accelerated video decoding with NVDEC ? Torchaudio 2.7.0 documentation","url":"https://docs.pytorch.org/audio/2.7.0/tutorials/nvdec_tutorial.html"},{"title":"Accelerated video decoding with NVDEC ? Torchaudio 2.2.0 documentation","url":"https://docs.pytorch.org/audio/2.2.0/tutorials/nvdec_tutorial.html"},{"title":"Accelerated video decoding with NVDEC ? Torchaudio 2.4.0 documentation","url":"https://docs.pytorch.org/audio/2.4.0/tutorials/nvdec_tutorial.html"}]

Based on the search results, I found relevant information about Decord and related video decoding technologies, though the specific maintenance status details you're asking about are limited. Here's what I found:

Decord Project

Decord supports NVDEC hardware accelerated decoding when compiled with `-DUSE_CUDA=ON` or similar CUDA configuration options. The GitHub repository for Decord is available at https://github.com/dmlc/decord.

Bridge to Torch

Decord includes a bridge feature that allows setting different backends, including PyTorch (`decord.bridge.set_bridge('torch')`), enabling conversion of decoded video frames to PyTorch tensors.

Recent Context (2024)

Starting with TorchAudio version 2.8, the project is transitioning into a maintenance phase, and APIs for video decoding are being deprecated and will be removed in 2.9, with the decoding capabilities being consolidated into TorchCodec.

The search results show that while Decord is actively documented and used as a video loading solution, specific maintenance status information regarding Decord's own project status for 2024-2025 was not directly found in these results. For the most current maintenance status of the Decord project specifically, I'd recommend checking the GitHub repository directly for recent commits, issues, and release information.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

25. user (2026-07-06T18:58:12.872Z)

Web search results for query: "torchvision 0.12.0 io VideoReader GPU device cuda decode support which version added deprecated"

Links: [{"title":"GPU decoder with 'cuda' backend not working as expected (shape / color / seek) · Issue #7745 · pytorch/vision","url":"https://github.com/pytorch/vision/issues/7745"},{"title":"torchvision.io.video_reader ? Torchvision 0.14 documentation","url":"https://docs.pytorch.org/vision/0.14/_modules/torchvision/io/video_reader.html"},{"title":"VideoReader ? Torchvision 0.14 documentation","url":"https://docs.pytorch.org/vision/0.14/generated/torchvision.io.VideoReader.html"},{"title":"RuntimeError: Not compiled with GPU decoder support. · Issue #6161 · pytorch/vision","url":"https://github.com/pytorch/vision/issues/6161"},{"title":"VideoReader ? Torchvision 0.15 documentation","url":"https://docs.pytorch.org/vision/0.15/generated/torchvision.io.VideoReader.html"},{"title":"torchvision.io.video_reader ? Torchvision 0.21 documentation","url":"https://docs.pytorch.org/vision/0.21/_modules/torchvision/io/video_reader.html"},{"title":"VideoReader ? Torchvision main documentation","url":"https://docs.pytorch.org/vision/main/generated/torchvision.io.VideoReader.html"},{"title":"torchvision.io.video_reader ? Torchvision main documentation","url":"https://docs.pytorch.org/vision/main/_modules/torchvision/io/video_reader.html"},{"title":"Accelerated video decoding on GPUs with CUDA and NVDEC ? TorchCodec 0.8.0+cu126 Documentation","url":"https://meta-pytorch.org/torchcodec/0.8/generated_examples/decoding/basic_cuda_example.html"},{"title":"VideoReader with cuda backend fails with num_workers > 1 · Issue

#8072 · pytorch/vision","url":"https://github.com/pytorch/vision/issues/8072"}]

Now let me search specifically for torchvision 0.12.0 release notes to find when GPU decode support and deprecation information was introduced:

Links: [{"title":"Releases · pytorch/vision","url":"https://github.com/pytorch/vision/releases"},{"title":"torchvision.io ? Torchvision 0.12 documentation","url":"https://docs.pytorch.org/vision/0.12/_modules/torchvision/io.html"},{"title":"Previous PyTorch Versions","url":"https://pytorch.org/get-started/previous-versions/"},{"title":"torchvision · PyPI","url":"https://pypi.org/project/torchvision/"},{"title":"torchvision ? Torchvision 0.23 documentation","url":"https://docs.pytorch.org/vision/0.23/_modules/torchvision.html"},{"title":"vision/torchvision/csrc/io/decoder/gpu/README.rst at main · pytorch/vision","url":"https://github.com/pytorch/vision/blob/main/torchvision/csrc/io/decoder/gpu/README.rst"},{"title":"Hardware-Accelerated Video Decoding and Encoding ? Torchaudio 0.13.1 documentation","url":"https://docs.pytorch.org/audio/0.13.1/hw_acceleration_tutorial.html"},{"title":"Hardware-Accelerated Video Decoding and Encoding ? Torchaudio 0.13.0 documentation","url":"https://docs.pytorch.org/audio/0.13.0/hw_acceleration_tutorial.html"},{"title":"Pytorch Vision","url":"https://docs.pytorch.org/vision/"},{"title":"GPU video decoder/encoder ? Torchaudio 2.0.1 documentation","url":"https://docs.pytorch.org/audio/2.0.1/hw_acceleration_tutorial.html"}]

Let me search more specifically for when GPU decoder support was first added and when it was deprecated:

Links: [{"title":"Releases · pytorch/vision","url":"https://github.com/pytorch/vision/releases"},{"title":"Previous PyTorch Versions","url":"https://pytorch.org/get-started/previous-versions/"},{"title":"torchvision · PyPI","url":"https://pypi.org/project/torchvision/"},{"title":"Releases · meta-pytorch/torchcodec","url":"https://github.com/meta-pytorch/torchcodec/releases"},{"title":"Add video GPU decoder by prabhat00155 · Pull Request #5019 · pytorch/vision","url":"https://github.com/pytorch/vision/pull/5019"},{"title":"torchvision 0.11.2","url":"https://pypi.org/project/torchvision/0.11.2/"},{"title":"Detected that PyTorch and torchvision were compiled with different CUDA versions - CUDA Setup and Installation - NVIDIA Developer Forums","url":"https://forums.developer.nvidia.com/t/detected-that-pytorch-and-torchvision-were-compiled-with-different-cuda-versions/267555"},{"title":"Pytorch compatibility issues (torch 2.0.0+nv23.5 && torchvision 0.15.1) - Jetson Orin NX - NVIDIA Developer Forums","url":"https://forums.developer.nvidia.com/t/pytorch-compatibility-issues-torch-2-0-0-nv23-5-torchvision-0-15-1/256116"},{"title":"What is the compatible torchvision version for torch-1.14.0a0+44dac51c.nv23.2 installed on AGX ORIN(Ubuntu-22.04, JP-5.1/5.0, Python-3.8, CUDA-11.4) - Jetson AGX Orin - NVIDIA Developer Forums","url":"https://forums.developer.nvidia.com/t/what-is-the-compatible-torchvision-version-for-torch-1-14-0a0-44dac51c-nv23-2-installed-on-agx-orin-ubuntu-22-04-jp-5-1-5-0-python-3-8-cuda-11-4/248828"},{"title":"RuntimeError: Not compiled with GPU decoder support. · Issue #6161 · pytorch/vision","url":"https://github.com/pytorch/vision/issues/6161"}]

Let me search for information about the specific deprecation timeline for torchvision video decoders:

Links: [{"title":"Decoding / Encoding images and videos ? Torchvision 0.22 documentation","url":"https://docs.pytorch.org/vision/0.22/io.html"},{"title":"Decoding / Encoding images and videos ? Torchvision main documentation","url":"https://docs.pytorch.org/vision/main/io.html"},{"title":"read_video ? Torchvision main documentation","url":"https://docs.pytorch.org/vision/main/generated/torchvision.io.read_video.html"},{"title":"torchvision.io.video_reader ? Torchvision main documentation","url":"https://docs.pytorch.org/vision/main/_modules/torchvision/io/video_reader.html"},{"title":"Decoding / Encoding images and videos ? Torchvision 0.24 documentation","url":"https://docs.pytorch.org/vision/0.24/io.html"},{"title":"VideoReader ? Torchvision main documentation","url":"https://docs.pytorch.org/vision/main/generated/torchvision.io.VideoReader.html"},{"title":"vision/torchvision/io/video.py at main · pytorch/vision","url":"https://github.com/pytorch/vision/blob/main/torchvision/io/video.py"},{"title":"torchvision.io.video_reader ? Torchvision 0.24 documentation","url":"https://docs.pytorch.org/vision/0.24/_modules/torchvision/io/video_reader.html"},{"title":"Decoding / Encoding images and videos ? Torchvision 0.23

documentation","url":"https://docs.pytorch.org/vision/0.23/io.html"},{"title":"write_video ? Torchvision main documentation","url":"https://docs.pytorch.org/vision/main/generated/torchvision.io.write_video.html"}]

Based on my search results, I can provide you with the following information about GPU decoder support in torchvision 0.12.0 and its deprecation:

GPU Decoder Support in torchvision 0.12.0

In torchvision 0.12.0, the VideoReader class supported GPU decoding via a `device` parameter that accepts "cuda", with the capability to create a GPUDecoder when CUDA device is specified.

When GPU Decoder Support Was Added

GPU video decoder support was added to torchvision's video reading API, and based on the available documentation, this feature appears to have been present in version 0.12.0.

Deprecation Timeline

All the video decoding and encoding capabilities of torchvision are deprecated from version 0.22 and will be removed in version 0.24. This means that while 0.12.0 included the GPU decoder functionality, it was later deprecated starting in version 0.22.

The video decoding and encoding utilities of TorchVision, which have been deprecated for a long time, are now removed, including torchvision.io.video.*, read_video, write_video, the VideoReader class, etc. (in version 0.26).

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

26. user (2026-07-06T18:58:15.726Z)

Web search results for query: "PyNvVideoCodec NVIDIA python NVDEC decode to CUDA tensor 2025 install pip supported CUDA versions"

Links: [{"title":"Read Me - NVIDIA Docs","url":"https://docs.nvidia.com/video-technologies/pynvvideocodec/read-me/index.html"},{"title":"PyNvVideoCodec API Programming Guide - NVIDIA Docs","url":"https://docs.nvidia.com/video-technologies/pynvvideocodec/pynvc-api-prog-guide/index.html"},{"title":"PyNvVideoCodec ? CV-CUDA Beta documentation","url":"https://cvcuda.github.io/CV-CUDA/interop/pynvvideocodec.html"},{"title":"PyNvVideoCodec - Get Started | NVIDIA Developer","url":"https://developer.nvidia.com/pynvvideocodec"},{"title":"v | January 2026 PyNvVideoCodec Read Me","url":"https://docs.nvidia.com/video-technologies/pynvvideocodec/pdf/Read_Me.pdf"},{"title":"GitHub - NVIDIA-AI-IOT/deepstream_libraries: DeepStream Libraries offer CVCUDA, NvImageCodec, and PyNvVideoCodec modules as Python APIs for seamless integration into custom frameworks. · GitHub","url":"https://github.com/NVIDIA-AI-IOT/deepstream_libraries"},{"title":"PyNvVideoCodec · PyPI","url":"https://pypi.org/project/PyNvVideoCodec/1.0.1/"},{"title":"GitHub - NVIDIA/VideoProcessingFramework: Set of Python bindings to C++ libraries which provides full HW acceleration for video decoding, encoding and GPU-accelerated color space and pixel format conversions · GitHub","url":"https://github.com/NVIDIA/VideoProcessingFramework"},{"title":"PyNvVideoCodec API Reference - NVIDIA Docs","url":"https://docs.nvidia.com/video-technologies/pynvvideocodec/pynvc-api-reference/index.html"},{"title":"What?s New in PyNvVideoCodec 2.0 for Python GPU-Accelerated Video Processing | NVIDIA Technical Blog","url":"https://developer.nvidia.com/blog/whats-new-in-pynvvideocodec-2-0-for-python-gpu-accelerated-video-processing/"}]

Now let me search for more specific information about supported CUDA versions and pip installation:

Links: [{"title":"v | January 2026 PyNvVideoCodec Read Me","url":"https://docs.nvidia.com/video-technologies/pynvvideocodec/pdf/Read_Me.pdf"},{"title":"PyNvVideoCodec - Get Started | NVIDIA

Developer","url":"https://developer.nvidia.com/pynvvideocodec"},{"title":"Sehwan Ki - PyNvCodec Example","url":"https://sites.google.com/site/sehwanki1004/pynvcodec-example"},{"title":"pynvvideocodec · PyPI","url":"https://pypi.org/project/pynvvideocodec/"},{"title":"GitHub - NVIDIA/VideoProcessingFramework: Set of Python bindings to C++ libraries which provides full HW acceleration for video decoding, encoding and GPU-accelerated color space and pixel format conversions · GitHub","url":"https://github.com/NVIDIA/VideoProcessingFramework"},{"title":"PyNvVideoCodec API Programming Guide - NVIDIA Docs","url":"https://docs.nvidia.com/video-technologies/pynvvideocodec/pynvc-api-prog-guide/index.html"},{"title":"pynvcodec · PyPI","url":"https://pypi.org/project/pynvcodec/"},{"title":"Read Me - NVIDIA Docs","url":"https://docs.nvidia.com/video-technologies/pynvvideocodec/read-me/index.html"},{"title":"CV-CUDA/samples/README.md at main · CVCUDA/CV-CUDA","url":"https://github.com/CVCUDA/CV-CUDA/blob/main/samples/README.md"}]

Let me search for more specific information about PyNvVideoCodec's CUDA version compatibility:

Links: [{"title":"CUDA Toolkit 13.3 Update 1 - Release Notes ? Release Notes 13.3 documentation","url":"https://docs.nvidia.com/cuda/cuda-toolkit-release-notes/index.html"},{"title":"v | January 2026 PyNvVideoCodec Read Me","url":"https://docs.nvidia.com/video-technologies/pynvvideocodec/pdf/Read_Me.pdf"},{"title":"pynvvideocodec · PyPI","url":"https://pypi.org/project/pynvvideocodec/"},{"title":"PyNvVideoCodec - Get Started | NVIDIA Developer","url":"https://developer.nvidia.com/pynvvideocodec"},{"title":"PyNvVideoCodec API Programming Guide - NVIDIA Docs","url":"https://docs.nvidia.com/video-technologies/pynvvideocodec/pynvc-api-prog-guide/index.html"},{"title":"GitHub - NVIDIA-AI-IOT/deepstream_libraries: DeepStream Libraries offer CVCUDA, NvImageCodec, and PyNvVideoCodec modules as Python APIs for seamless integration into custom frameworks. · GitHub","url":"https://github.com/NVIDIA-AI-IOT/deepstream_libraries"},{"title":"CUDA Toolkit 13.1 - Release Notes ? Release Notes 13.1 documentation","url":"https://docs.nvidia.com/cuda/archive/13.1.0/cuda-toolkit-release-notes/index.html"},{"title":"Profile of nvidia · PyPI","url":"https://pypi.org/user/nvidia/"},{"title":"Releases · CVCUDA/CV-CUDA","url":"https://github.com/CVCUDA/CV-CUDA/releases"},{"title":"CUDA Toolkit 13.2 - Release Notes ? Release Notes 13.2 documentation","url":"https://docs.nvidia.com/cuda/archive/13.2.0/cuda-toolkit-release-notes/index.html"}]

Based on my web search results, here's comprehensive information about PyNvVideoCodec:

PyNvVideoCodec Overview

PyNvVideoCodec is NVIDIA's Python based video codec library that provides simple yet powerful Python APIs for hardware accelerated video encode and decode on NVIDIA GPUs. PyNvVideoCodec is the successor of VPF (Video Processing Framework).

NVDEC Decoding to CUDA Tensors

Using the SimpleDecoder API, frames are decoded directly to GPU memory in RGB format and converted to PyTorch tensors for downstream processing. The library supports multiple tensor conversion approaches:

- PyNvVideoCodec implements the Python DLPack protocol through __dlpack__() and __dlpack_device__() methods on decoded frames. This allows seamless integration with any framework that supports the DLPack protocol.
- CUDA Array Interface support to facilitate data exchange with NVIDIA's CV-CUDA library.

Installation via pip

PyNvVideoCodec library is distributed in two formats: binary distribution via PyPl and source code distribution via NVIDIA NGC. In both cases, it can be installed using a single pip install command. The ready-to-use Python WHL's (Wheel) of the PyNvVideoCodec for Windows, Linux and WSL (Windows Subsystem for Linux) are hosted on PyPI. This is the recommended way to install PyNvVideoCodec.

Python Version Support

Requires: Python >=3.8

Supported Platforms

PyNvVideoCodec is supported on Windows, Linux, and Windows Subsystem for Linux (WSL).

CUDA Compatibility

While the search results don't explicitly list specific CUDA version requirements, the documentation indicates a compatible CUDA toolkit must be installed for PyCUDA. The library is distributed as pre-built wheels for common platforms, and all these packages are built against the same CUDA version and validated with the specified driver version.

Key Features

Current PyNvVideoCodec version supports following features: Codecs: H.264, HEVC, AV1. Surface formats: NV12 (8 bit), YUV 4:2:0 (10 bit), YUV 4:4:4 (8 and 10 bit) ... DLPack support to facilitate data exchange with popular DL frameworks like PyTorch and TensorRT.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

27. assistant (2026-07-06T18:58:22.268Z)

Good baseline. Now let me nail down the specifics that matter most for ranking: (1) DALI's CUDA 11.x wheel + torch 1.11 DLPack interop and whether it forward-compatible-runs on a 12.x driver; (2) torchvision 0.12 GPU decoder's known correctness bugs (color/seek), which directly threatens parity; (3) OpenCV cudacodec build friction on pip vs the need to compile; (4) PyAV / ffmpeg h264_cuvid CPU-roundtrip reality. Let me search these.

28. user (2026-07-06T18:58:29.488Z)

Web search results for query: "DALI CUDA 11.0 build "enhanced compatibility" run on newer driver 12.x forward compatible cuda110 wheel pip install"

Links: [{"title":"Releases · NVIDIA/DALI","url":"https://github.com/NVIDIA/DALI/releases"},{"title":"CUDA Compatibility Release r595 NVIDIA Corporation Mar 31, 2026","url":"https://docs.nvidia.com/deploy/pdf/CUDA_Compatibility.pdf"},{"title":"Installation ? NVIDIA DALI","url":"https://docs.nvidia.com/deeplearning/dali/user-guide/docs/installation.html"},{"title":"Forward Compatibility ? CUDA Compatibility","url":"https://docs.nvidia.com/deploy/cuda-compatibility/forward-compatibility.html"},{"title":"Installation ? NVIDIA DALI 1.1.0 documentation","url":"https://docs.nvidia.com/deeplearning/dali/archives/dali_110/user-guide/docs/installation.html"},{"title":"CUDA Compatibility ? CUDA Compatibility","url":"https://docs.nvidia.com/deploy/cuda-compatibility/latest/"},{"title":"Compiling DALI from Source ? NVIDIA DALI","url":"https://docs.nvidia.com/deeplearning/dali/user-guide/docs/compilation.html"},{"title":"Supported NVIDIA hardware, CUDA, OS, and CUDA driver ? NVIDIA DALI","url":"https://docs.nvidia.com/deeplearning/dali/user-guide/docs/support_matrix.html"},{"title":"NVIDIA/DALI v1.36.0 on GitHub","url":"https://newreleases.io/project/github/NVIDIA/DALI/release/v1.36.0"},{"title":"CUDA Compilers · GitHub","url":"https://gist.github.com/ax3l/9489132"}]

Based on the search results, here's what I found regarding DALI with CUDA 11.0, enhanced compatibility, and driver compatibility:

DALI CUDA 11.0 Enhanced Compatibility

CUDA 11.0 builds use CUDA toolkit enhanced compatibility, and are built with the latest CUDA

11.x toolkit but can run on the latest, stable CUDA 11.0 capable drivers (450.80 or later). This means DALI wheels built for CUDA 11.0 are forward compatible with newer drivers in the 11.x and beyond range.

Installation via pip

You can install the CUDA 11.0 build using: `pip install --upgrade nvidia-dali-cuda110` or `pip install --extra-index-url https://developer.download.nvidia.com/compute/redist/cuda/11.0 --upgrade nvidia-dali`.

Driver Compatibility

The enhanced compatibility feature means that DALI wheels can run on drivers from 450.80 and later, though using the most recent driver may enable additional functionality. This provides compatibility across CUDA driver versions while allowing for potential improvements with newer drivers.

The `nvidia-dali-cuda110` wheel package format is specifically designed to work with CUDA 11.0+ compatible drivers, supporting the enhanced compatibility mode NVIDIA introduced to reduce driver upgrade requirements.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

29. user (2026-07-06T18:58:30.831Z)

Web search results for query: "opencv cudacodec VideoReader pip wheel not included build from source CUDA nvcuvid required 2024"

Links: [{"title":"Enable cudacodec::VideoReader - OpenCV Q&A Forum","url":"https://answers.opencv.org/question/188525/enable-cudacodecvideoreader/"},{"title":"Releases · cudawarped/opencv-python-cuda-wheels","url":"https://github.com/cudawarped/opencv-python-cuda-wheels/releases"},{"title":"Unable to enable Cudacodec VideoReader · Issue #11220 · opencv/opencv","url":"https://github.com/opencv/opencv/issues/11220"},{"title":"OpenCV: cv::cudacodec::VideoReader Class Reference","url":"https://docs.opencv.org/4.x/db/ded/classcv_1_1cudacodec_1_1VideoReader.html"},{"title":"cv::cudacodec::VideoWriter not working with only NVCUVID - C++ - OpenCV","url":"https://forum.opencv.org/t/cv-videowriter-not-working-with-only-nvcuvid/16009"},{"title":"? Incompatibility of CUDA CMake options · Issue #25920 · opencv/opencv","url":"https://github.com/opencv/opencv/issues/25920"},{"title":"cv::cudacodec::VideoReader throw_no_cuda · Issue #21726 · opencv/opencv","url":"https://github.com/opencv/opencv/issues/21726"},{"title":"DNN: CUDA backend requires cuDNN. Please resolve dependency or disable OPENCV_DNN_CUDA=OFF · Issue #25426 · opencv/opencv","url":"https://github.com/opencv/opencv/issues/25426"},{"title":"cv::cudacodec::createVideoReader work well while cv::cudacodec::createVideoWriter cause ERROR - Video Processing & Optical Flow - NVIDIA Developer Forums","url":"https://forums.developer.nvidia.com/t/cv-createvideoreader-work-well-while-cv-createvideowriter-cause-error/175045"}]

Based on the search results, here's what I found regarding OpenCV's cudacodec VideoReader and the pip wheel/source build issue:

Key Findings:

Standard pip wheels don't include cudacodec support: The standard OpenCV pip packages do not include NVCUVID/NVCUVENC support by default. cudacodec::VideoReader requires Nvidia Video Codec SDK and cudacodec::VideoWriter requires Nvidia Video Codec SDK.

Building from source is necessary: To use cudacodec::VideoReader, you need to build OpenCV from source with CUDA support and the Nvidia Video Codec SDK.

Alternative: Pre-built CUDA wheels: There are pre-compiled wheels available that include

NVIDIA CUDA with NVCUVID and NVCUVENC support. The `opencv-python-cuda-wheels` project provides builds with these components enabled.

CMake configuration: When building from source, you need to ensure the proper CMake flags are set. You will need nvcuvid and nvcuvenc libraries configured during the build process.

Common error: Users often encounter an error about functionality being "disabled for current build or platform" when trying to use cudacodec on a standard pip installation, which indicates the feature wasn't compiled in.

The bottom line is that if you need CUDA-accelerated video decoding via cudacodec, you'll either need to build OpenCV from source with the proper NVIDIA Video Codec SDK installed, or use pre-built CUDA-enabled wheels from community projects like `cudawarped/opencv-python-cuda-wheels`.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

30. user (2026-07-06T18:58:31.558Z)

Web search results for query: "torchvision GPU VideoReader cuda backend color wrong shape bug issue 7745 seek reliability sequential"

Links: [{"title":"GPU decoder with 'cuda' backend not working as expected (shape / color / seek) · Issue #7745 · pytorch/vision","url":"https://github.com/pytorch/vision/issues/7745"},{"title":"GPU VideoReader not working","url":"https://lightrun.com/answers/pytorch-vision-gpu-videoreader-not-working"},{"title":"VideoReader with cuda backend fails with num_workers > 1 · Issue #8072 · pytorch/vision","url":"https://github.com/pytorch/vision/issues/8072"},{"title":"GPU VideoReader not working · Issue #5702 · pytorch/vision","url":"https://github.com/pytorch/vision/issues/5702"},{"title":"torchvision.io.video_reader ? Torchvision main documentation","url":"https://docs.pytorch.org/vision/main/_modules/torchvision/io/video_reader.html"},{"title":"VideoReader ? Torchvision main documentation","url":"https://docs.pytorch.org/vision/main/generated/torchvision.io.VideoReader.html"},{"title":"RuntimeError: cuda video backend is not available. · Issue #8166 · pytorch/vision","url":"https://github.com/pytorch/vision/issues/8166"},{"title":"Torchvision fails when trying to use the gpu - vision - PyTorch Forums","url":"https://discuss.pytorch.org/t/torchvision-fails-when-trying-to-use-the-gpu/176898"},{"title":"2022: state of video IO in torchvision · Issue #5720 · pytorch/vision","url":"https://github.com/pytorch/vision/issues/5720"}]

Based on the search results, I found information about Issue #7745 on the PyTorch Vision GitHub repository, which relates to problems with the CUDA backend in torchvision's VideoReader.

Summary of Issue #7745

The issue describes problems with the CUDA backend for the VideoReader object, where it doesn't work as expected. The problems identified include issues with frame shape, color format, and seek reliability.

From the search results, I can see that this is one of several known issues with the GPU video decoding implementation. The CUDA video decoder is still in BETA stage with many edge-cases that need to be figured out.

The GitHub issue appears to document multiple problems users have encountered when trying to use the CUDA backend for GPU-accelerated video decoding, including problems with frame dimensions, color space conversions, and sequential frame reading reliability when using the seek function.

Additionally, all video decoding and encoding capabilities of torchvision are deprecated from version 0.22 and will be removed in version 0.24, with migration to TorchCodec recommended.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

31. user (2026-07-06T18:58:33.366Z)

Web search results for query: "PyAV ffmpeg h264_cuvid hwaccel cuda frames download to CPU numpy GPU surface av 2024"

Links: [{"title":"AV1 support for HW decoding · PyAV-Org/PyAV · Discussion #1691","url":"https://github.com/PyAV-Org/PyAV/discussions/1691"},{"title":"Set hwaccel ? · Issue #307 · PyAV-Org/PyAV","url":"https://github.com/mikeboers/PyAV/issues/307"},{"title":"How to make sure hardware acceleration is used by PyAV? · PyAV-Org/PyAV · Discussion #1821","url":"https://github.com/PyAV-Org/PyAV/discussions/1821"},{"title":"avcuda · PyPI","url":"https://pypi.org/project/avcuda/"},{"title":"Use FFmpeg to Decode H.264 Stream with NVIDIA GPU Acceleration | by zong fan | Medium","url":"https://medium.com/@fanzongshaoxing/use-ffmpeg-to-decode-h-264-stream-with-nvidia-gpu-acceleration-16b660fd925d"},{"title":"Using h264_cuvid as hardware decoder · Issue #451 · PyAV-Org/PyAV","url":"https://github.com/mikeboers/PyAV/issues/451"},{"title":"Preserving hardware memory during cuvid decoding, exporting/importing via dlpack. by caffeinism · Pull Request #2155 · PyAV-Org/PyAV","url":"https://github.com/PyAV-Org/PyAV/pull/2155"},{"title":"NVIDIA FFmpeg Transcoding Guide | NVIDIA Technical Blog","url":"https://developer.nvidia.com/blog/nvidia-ffmpeg-transcoding-guide/"}]

Based on the search results, here's what I found regarding PyAV, FFmpeg H.264 CUVID hardware acceleration, and GPU/CPU memory transfers:

Key Findings:

Hardware Acceleration with PyAV:

PyAV supports hardware acceleration using `HWAccel` with CUDA device type, and you can verify it's enabled by checking `codec_context.is_hwaccel`.

H.264 CUVID Decoder:

When using FFmpeg with hwaccel, you can specify hardware-accelerated decoders like h264_cuvid for CUDA-based decoding.

GPU Memory Management:

Unless `-hwaccel_output_format cuda` is explicitly specified, FFmpeg CLI automatically performs a hardware-to-software transfer (hwdownload) to move frames from VRAM to system RAM. Using `-hwaccel cuda -hwaccel_output_format cuda` can result in up to 2x throughput compared to unoptimized calls.

Recent PyAV Developments (2024):

A recent PR in PyAV added support for preserving hardware memory during CUVID decoding and exporting frames via DLPack, allowing conversion to PyTorch tensors on CUDA devices. PyAV-CUDA is an extension providing hardware-accelerated video decoding with Nvidia GPUs and CUDA-accelerated kernels for color space conversion.

Challenges:

The hardware decode pipeline in PyAV hasn't been very optimized, so speed may not match FFmpeg CLI.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

32. assistant (2026-07-06T18:58:38.857Z)

I now have strong signal on all six candidates. Let me verify the two most decision-critical facts directly from primary sources: (1) whether torchvision **0.12.0** was even *built* with GPU decoder support in the standard PyTorch cu113 wheel (the feature existed in the API but the wheels historically weren't compiled with it ? issue #6161), and (2) confirm the `opencv-python-

cuda-wheels` availability for the older CUDA/py3.8 combo. Let me fetch the two key GitHub issues.

33. user (2026-07-06T18:58:45.265Z)

Based on the GitHub issue content provided, I cannot find a maintainer response that directly addresses whether official PyPI wheels include GPU video decoder support.

The issue shows a user experiencing the error "Not compiled with GPU decoder support" when attempting to use `VideoReader` with CUDA. The user compiled torchvision from source following specific build instructions that involved setting environment variables for the NVIDIA Video Codec SDK.

However, **the page does not contain any maintainer commentary or answers** about whether released wheels have GPU NVDEC decoding compiled in. The issue appears to be open without resolution details visible in the provided content. To get a definitive answer on this topic, one would need to read the full discussion thread or the official torchvision documentation.

34. user (2026-07-06T18:58:46.171Z)

nvidia-dali-cuda110 Support and Installation

Python Versions Supported

The package requires "Python <3.14, >=3.9", meaning it supports Python 3.9 through 3.13. **Python 3.8 is not supported.**

CUDA Version

This package is specifically built for CUDA 11.0, as indicated by the "cuda110" suffix in the package name.

Latest Version

Version 1.50.0, released May 27, 2025.

Installation Command

```bash
pip install nvidia-dali-cuda110
```

The package is available on PyPI at `https://pypi.org/project/nvidia-dali-cuda110/`. No custom index URL is needed for standard installation from the official Python Package Index.

35. assistant (2026-07-06T18:58:54.929Z)

That PyPI mirror reflects the latest version's metadata (which dropped py3.8). The DALI docs said 1.49 was the last with py3.8 support, and DALI ships from NVIDIA's own index. Let me confirm the torchvision GPU-decoder-not-in-wheel fact from the torchvision GPU README (the authoritative source), and confirm decord's maintenance status directly from its repo. These are the two remaining load-bearing facts.

36. user (2026-07-06T18:58:57.965Z)

The server returned HTTP 404 Not Found.

The response body was not retrieved. If this URL requires authentication, use an authenticated tool (e.g. `gh` for GitHub, or an MCP-provided fetch tool) instead of WebFetch.

37. user (2026-07-06T18:59:04.608Z)

Decord Project Status Analysis

Based on the provided GitHub page content, I cannot definitively answer your questions because:

Missing Information

Most Recent Commit & Release Dates: The page shows "352 Commits" and "15 tags" but does not display specific dates for the most recent activity.

Project Maintenance Status: There is insufficient timestamp data to determine if the project is actively maintained in 2024-2025 or abandoned.

GPU Wheel Availability: The documentation explicitly states: "Note that only CPU versions are provided with PYPI now. Please build from source to enable GPU acclerator." This indicates GPU support requires building from source rather than using pip wheels.

CUDA & Python Version Support: The page does not specify which CUDA versions or Python versions are supported by the wheels.

What Can Be Confirmed

- The project supports GPU acceleration via NVIDIA NVDEC, but only through source compilation
- Installation requires either pip (CPU only) or manual compilation from source
- The repository is organized with active CI/CD workflows, though their current status isn't shown in this excerpt

To determine current maintenance status and version compatibility, you would need to check the repository's commit history, release dates, and documentation files directly on GitHub.

38. assistant (2026-07-06T18:59:09.119Z)

Confirmed the key decord fact: **PyPI provides CPU-only; GPU requires source build**. Let me pin down decord's maintenance status (last release/commit dates) and confirm the torchvision GPU-decoder build-from-source requirement via the correct README path and the PR that added it. These two facts finalize the ranking.

39. user (2026-07-06T18:59:16.656Z)

Web search results for query: "decord last release 0.6.0 date github stale unmaintained 2023 2024 no new releases NVDEC eva-decord fork"

Links: [{"title":"GitHub - dmlc/decord: An efficient video loader for deep learning with smart shuffling that's super easy to digest · GitHub","url":"https://github.com/dmlc/decord"},{"title":"GitHub - georgia-tech-db/eva-decord: An efficient video loader for deep learning with smart shuffling that's super easy to digest · GitHub","url":"https://github.com/georgia-tech-db/eva-decord"},{"title":"Releases · dmlc/decord","url":"https://github.com/dmlc/decord/releases?after=v0.6.0"},{"title":"decord 0.6.0 on PyPI - Libraries.io - security & maintenance data for open source software","url":"https://libraries.io/pypi/decord"},{"title":"decord/README.md at master · dmlc/decord","url":"https://github.com/dmlc/decord/blob/master/README.md"},{"title":"GitHub - DutchErwin/decord-ffmpeg7: An efficient video loader for deep learning with smart shuffling that's super easy to digest","url":"https://github.com/DutchErwin/decord-ffmpeg7"},{"title":"GitHub - mrezabayat/decord2: An efficient video loader for deep learning with smart shuffling that's super easy to digest · GitHub","url":"https://github.com/mrezabayat/decord2"},{"title":"GitHub - johnnynunez/decord2: An efficient video loader for deep learning with smart shuffling that's super easy to digest · GitHub","url":"https://github.com/johnnynunez/decord2"},{"title":"eva-decord 0.6.1 on PyPI - Libraries.io - security & maintenance data for open source software","url":"https://libraries.io/pypi/eva-decord"},{"title":"decord · PyPI","url":"https://pypi.org/project/decord/"}]

Based on the search results, here's what I found about the decord project:

Release Status

The latest release of decord is v0.6.0, released on June 14 (based on the GitHub releases page showing "14 Jun 20:44"). This was a pre-release that fixed stability and tensorflow bridge

issues.

Maintenance Status

The search results confirm that decord has not had new releases since v0.6.0. Based on the release history visible in the search results, there is no evidence of newer releases beyond v0.6.0.

Alternative Projects

The search results show that there are several fork and alternative projects available:

1. **eva-decord**: This is a fork of Decord that supports ffmpeg5 and builds/installs on newer macos versions.

2. **Other forks**: The search revealed additional community forks including `decord-ffmpeg7` and other variations maintained by different developers.

The original decord repository at `dmlc/decord` does support NVDEC hardware acceleration for video decoding, as noted in the documentation for using the `-DUSE_CUDA=ON` flag during builds.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

40. user (2026-07-06T18:59:25.711Z)

GPU Decoder Distribution in Torchvision

Based on the PR content provided, there is **no explicit statement about whether GPU decoding is included in standard pip wheels**. The discussion focuses on implementation details rather than distribution mechanics.

What the PR Shows

The PR demonstrates that GPU decoding requires:
- NVIDIA CUDA toolkit integration
- FFmpeg libraries with specific headers (checking for `bsf.h`)
- NVIDIA Video Codec SDK (implied through CUDA-specific decoder implementations)

Key Development Notes

The reviewer noted: "in a follow-up PR we will need to add CI tests for those functions. This might be a bit involved, but looks like the NVIDIA-docker has nvdec available in the container"?suggesting GPU decoding support may require special build configurations rather than being universally available.

Version Information

This PR was merged on **December 30, 2021** into the `main` branch, which would have shipped in a subsequent release (likely 0.13.0 or later), but the exact version is not specified in this document.

Conclusion: The PR content does not definitively state whether released binaries include GPU decoding by default, only that the feature was added to support GPU-accelerated video reading when properly configured.

41. assistant (2026-07-06T19:01:05.062Z)

I have all the load-bearing facts confirmed from primary sources. Key findings solidified:

- **decord**: last release v0.6.0 (2022), effectively stale; PyPI ships **CPU-only** ? GPU/NVDEC

needs a from-source build. Community forks (eva-decord, decord2) exist but none ship GPU pip wheels.
- **torchvision GPU decoder**: merged Dec 2021, first shipped ~0.13; NOT compiled into standard PyPI wheels (needs source build with Video Codec SDK), known correctness bugs (color/shape/seek, #7745), and **0.12.0 predates it entirely** ? so it's a non-starter for the pinned env even before the parity concern.
- **DALI**: last py3.8-supporting release is 1.49; CUDA 11.0 build uses enhanced compatibility (runs on driver ?450.80, so fine under driver 580); ships as ready pip wheels from NVIDIA's index.
- **OpenCV cudacodec**: not in standard wheels; needs source build OR the community `cudawarped/opencv-python-cuda-wheels`.
- **PyNvVideoCodec**: NVIDIA-maintained, py?3.8, ready PyPI wheels, DLPack?torch, decodes to GPU; but frames land as NV12/RGB on GPU (color-space handling is the parity risk).
- **ffmpeg/PyAV h264_cuvid**: without `-hwaccel_output_format cuda` it auto-hwdownloads to host RAM (CPU round-trip), which is the drop-in, parity-safe option.

Now I have everything needed. Let me write the deliverable.

Research Spike: NVDEC decode backend for WASB shuttle-detector (issue #349)

1. The loader contract the decode path must satisfy

Read from `backend/pipeline/detectors/wasb_infer.py` (absolute: `C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\pipeline\detectors\wasb_infer.py`):

Element	Location	Contract
`make_video_frame_loader`	L743?783	Returns `(frame_loader, count_hint, release)`. `frame_loader(index) -> BGR uint8 ndarray`, **forward-only**, `cv2.VideoCapture` + `cap.set(CAP_PROP_POS_FRAMES)` seek only on resume.
`SequentialFrameStore`	L269?316	Sliding buffer, window = `frames_in`; calls `reader(index)` with **non-decreasing** indices; keeps O(window) frames.
Frame?tensor	`_bgr_to_pil_rgb` L328?340	`cv2.cvtColor(bgr, COLOR_BGR2RGB)` ? `PIL.Image.fromarray` ? RGB PIL image. Engineered to be **byte-identical** to the PNG-on-disk path (`cv2.imwrite` PNG ? `Image.open().convert('RGB')`), "verified empirically, max|diff|=0".
`read_image` shim	L343?397	Rebinds `dataloaders.dataset_loader.read_image` to feed the in-memory PIL frame into WASB's **unmodified** `ImageDataset.__getitem__`.
Resize + normalize	inside WASB repo (`build_img_transforms`/`ImageDataset`, external)	**Resize to `inp_width×inp_height` happens on the CPU PIL frame, downstream of the loader.** The loader hands off full-resolution BGR.
Batching	`DataLoader(shuffle=False, num_workers=0)` + `_PrefetchIter` L403?445	Streaming forces `num_workers=0` (in-process shim can't be pickled). `_PrefetchIter` overlaps CPU batch-assembly with GPU inference via one producer thread.
GPU hand-off	`detector.run_tensor(imgs, trans)` L646	Receives a **CPU** tensor batch (assembled by the DataLoader from PIL via WASB transforms) + affine `trans`; moves `imgs` to GPU internally.

Two constraints dominate the whole evaluation:

1. **Parity boundary is above the loader.** The contract is "return a BGR uint8 ndarray identical to what a lossless PNG round-trip yields." Any NVDEC backend produces YUV/NV12 that must be color-converted to **BGR bit-identically to `cv2`'s H.264 SW-decode path** ? the hard part. NVDEC's built-in YUV?RGB (BT.601/709 matrix, limited vs. full range, chroma upsampling) will **not** match libavcodec/cv2 pixel-for-pixel. This is the central risk, not raw compatibility.
2. **The GPU tensor never reaches `run_tensor` from the loader.** The loader returns a host ndarray; resize/ToTensor run on CPU inside WASB; only then does WASB move to GPU. So "keep frames on GPU" gives **no** end-to-end win *without also rewriting WASB's transform/collate path* ? which is out of scope ("Do NOT change code") and would blow the parity guarantee. An NVDEC backend that decodes on GPU then `hwdownload`s to a host BGR ndarray is the only drop-in shape.

2. Compatibility table (pinned env: torch 1.11.0+cu113, torchvision 0.12.0+cu113, py3.8, conda/micromamba o

Backend	Works w/ py3.8 + cu113 torch 1.11?	Install friction (conda/micromamba)	Frames stay on GPU?	Sequential + resize + CUDA-tensor fit	Maintenance	Parity risk
ffmpeg `-hwaccel cuda`/`h264_cuvid`, hwdownload ? host BGR (subprocess pipe or PyAV)	? Independent of torch; ffmpeg already in image	? Low ? ffmpeg present; `av` is a pip wheel; or pure subprocess	? By design downloads to host (that's the point for drop-in)	?? Best ? returns BGR ndarray, WASB resizes as today	? ffmpeg active; PyAV active	**Medium** ? NVDEC YUV?BGR ? cv2 SW-decode exactly; **must parity-gate**
NVIDIA DALI (`fn.experimental.decoders.video` / video reader)	?? py3.8 only ? DALI **1.49**; use `nvidia-dali-cuda110` (CUDA 11.0 enhanced-compat, driver ?450.80 ? fine on 580)	?? Medium ? pinned old wheel from NVIDIA index; heavy dep	? Yes (GPU pipeline)	?? Pipeline/graph model, not a random-seek `reader(i)`; sequential OK but seek-resume awkward; DLPack?torch	? Active (but 1.49 branch frozen for py3.8)	**High** ? its own color/resize; hard to match cv2
PyNvVideoCodec (NVIDIA NVDEC bindings)	? py?3.8, ready PyPI wheels; runtime-linked to driver (580 ok)	? Low ? `pip install PyNvVideoCodec`	? Yes; DLPack/CAI ? torch	?? Decode API is stream/seek-capable; you own YUV?RGB?BGR + download	? NVIDIA-maintained (VPF successor)	**High** ? you implement color conversion; matching cv2 is on you
OpenCV `cv2.cudacodec.VideoReader`	?? Not in standard `opencv-python`; env already has **opencv-python-headless** (no CUDA). Need source build or `cudawarped/opencv-python-cuda-wheels`; must match py3.8 + a CUDA the wheel was built against	? High ? replace the pinned headless build; risk clashing with WASB's cv2 usage	? `nextFrame()` returns a `GpuMat`; download to get ndarray	?? API close to cv2 but GpuMat, not ndarray	?? Community wheels (cudawarped) active; core feature stable	**High** ? cudacodec color path ? cv2 CPU path
torchvision.io GPU `VideoReader(device="cuda")`	? **GPU decoder merged Dec 2021, first shipped ~0.13; `0.12.0` predates it.** Even later, **not compiled into PyPI wheels** (needs source build w/ Video Codec SDK). Known color/shape/**seek** bugs (#7745). Deprecated 0.22, removed 0.26	? Very high ? would force torch/torchvision bump = **env bump**	? (when built)	? seek unreliable (#7745) ? breaks resume	? Deprecated ? removed; superseded by TorchCodec	**High** + broken
decord (GPU)	? PyPI ships **CPU-only**; GPU needs `-DUSE_CUDA=ON` **source build**. Last release **v0.6.0 (2022)** ? effectively unmaintained	? High ? source build; forks (eva-decord, decord2) don't ship GPU wheels either	? (when built) `bridge_to_torch`	?? has seek, but stale	? Stale since 2022	**High**

3. Ranked recommendation

Primary: ffmpeg NVDEC with `hwdownload` to a host BGR ndarray (behind `decode_backend="nvdec"`, default `"cpu"`), preferring a **subprocess `ffmpeg` pipe of `rawvideo bgr24`** over PyAV.

Rationale:
- Zero impact on the pinned wheel set ? it's a separate process / a pip `av` wheel; **cannot force an env bump.** ffmpeg is already installed in both deploy images.
- Maps cleanly onto the loader: it still returns a **BGR uint8 ndarray**, so `SequentialFrameStore` ? `_bgr_to_pil_rgb` ? WASB transforms/resize run **completely unchanged**. This is the only option that preserves the byte-parity architecture without touching WASB.
- Forward-only sequential read is exactly ffmpeg's happy path; resume = restart the pipe with `-ss`/frame-select at `start_index`.
- The GPU?host copy is deliberate and acceptable: the WASB path already needs a host frame (resize is CPU-side), and NVDEC still offloads the expensive H.264 entropy/motion-comp decode from CPU to the L4's dedicated decoder block.

Fallback: PyNvVideoCodec (NVIDIA-maintained, clean py3.8 wheel, driver-linked so the cu113/driver-580 split is irrelevant to it). Choose this only if profiling shows the hwdownload copy is the bottleneck and you're willing to own a GPU-resident path ? which requires rewriting WASB's collate/transform to accept a CUDA tensor and **re-proving parity**, i.e. more than a loader swap.

Explicitly rejected for this env:
- **torchvision GPU VideoReader** ? not in 0.12.0 at all; would force a torch/torchvision bump

(**env-bump flag**), and has unreliable seek (#7745) that breaks resume.
- **decord GPU** ? CPU-only wheels, source build required, unmaintained since 2022.
- **OpenCV cudacodec** ? requires ripping out the pinned `opencv-python-headless` for a CUDA build; high risk of clashing with WASB's own cv2 use.
- **DALI** ? frozen on 1.49 for py3.8, and its graph/own-color model fights both the `reader(i)` seek contract and the byte-parity requirement.

4. Integration sketch against `wasb_infer.py`'s loader (default-OFF, parity-gated)

Add a factory that dispatches on a new `decode_backend` flag; **do not touch** `SequentialFrameStore`, `_bgr_to_pil_rgb`, or the shim ? the whole point is that the NVDEC loader satisfies the *identical* `frame_loader(index) -> BGR uint8 ndarray` contract.

```python
# new, additive ? sits beside make_video_frame_loader (L743)
def make_video_frame_loader(video_path, decode_backend="cpu"):   # default cpu
    if decode_backend == "cpu":
        return _make_cpu_frame_loader(video_path)        # today's cv2 body, unchanged
    elif decode_backend == "nvdec":
        return _make_nvdec_frame_loader(video_path)      # ffmpeg -hwaccel cuda, bgr24 out
    raise SystemExit(f"unknown decode_backend {decode_backend!r}")
```

`_make_nvdec_frame_loader` returns the same 3-tuple `(frame_loader, count_hint, release)`:
- `count_hint`: keep `cv2.CAP_PROP_FRAME_COUNT` probe (`_probe_frame_count`, L731) so the frame list is identical.
- `frame_loader(index)`: hold a long-lived `ffmpeg -hwaccel cuda -hwaccel_output_format cuda -i ? -vf "hwdownload,format=bgr24" -f rawvideo -pix_fmt bgr24 pipe:1` process; read `H*W*3` bytes ? `np.frombuffer(...).reshape(H, W, 3)`. Forward-only reads pull the next frame; a resume (`index > pos`) re-spawns the process seeking to `index` (or select frames), mirroring the existing `cap.set(POS_FRAMES)` seek-once semantics (L770?773).
- `release()`: terminate the ffmpeg process.

Thread the `decode_backend` param up through `run_video_streaming` (L786) ? `make_video_frame_loader`, defaulting `"cpu"` everywhere, and expose it as a CLI/set flag. Nothing downstream of the loader changes.

Parity gate (mandatory before default-ON): a test that decodes N frames both ways and asserts the WASB **network-input tensors** match within tolerance (a small ?, not `max|diff|=0` ? NVDEC color conversion will differ by a few LSB from cv2's SW path). Gate default-ON on either byte-parity OR an end-to-end trajectory-CSV equivalence within the detector's tolerance. **If exact parity fails**, the flag stays opt-in and any consumer must accept a documented pixel delta.

5. Top risks / open questions

1. **cu113-vs-driver split is a non-issue for the primary pick, real for others.** ffmpeg and PyNvVideoCodec link NVDEC through the **driver** (580, CUDA 12.x) at runtime ? independent of the wasb env's cu113 torch. DALI's CUDA-11.0 enhanced-compat wheel also runs on driver ?450.80. Only torchvision-GPU / cudacodec / decord-GPU would drag CUDA into the *build*, and those are already rejected. **No candidate in the recommendation forces an env bump ? flag confirmed clear.**
2. **Color/chroma parity is THE risk.** NVDEC YUV?RGB (BT.601 vs 709, limited vs full range, chroma-siting/upsampling) will not match libavcodec's SW decode (what `cv2.VideoCapture` uses) bit-for-bit. The repo's parity guarantee (`max|diff|=0`) almost certainly **will not hold**; the realistic bar is "detector output within tolerance." Open question: is the detector's trajectory output stable under a few-LSB pixel delta? Needs the parity harness to answer.
3. **hwdownload negates the "stay on GPU" benefit ? accepted deliberately.** True GPU-resident hand-off would need WASB's transform/collate rewritten to resize on-GPU and pass a CUDA tensor to `run_tensor` ? out of scope and parity-breaking. Flag if a future issue wants the fully-GPU path.
4. **Codec coverage.** Confirm source clips are H.264/HEVC that L4 NVDEC supports (it does for H.264/HEVC/AV1); VP9/odd profiles or B-frame/variable-fps edge cases can desync frame indexing

vs. the `cv2` count. Validate `count_hint` equals actual decoded frames on a sample.
5. **Resume seek exactness.** The CPU path relies on `cap.set(CAP_PROP_POS_FRAMES)` frame-accurate seek; ffmpeg `-ss` is timestamp-based and can land off-by-frames on VFR/open-GOP content. The NVDEC loader must seek **frame-accurately** (decode-and-discard to `index`, or `-ss` before `-i` + trim) to keep resume identical.
6. **Wheel availability drift.** `nvidia-dali-cuda110`'s current PyPI metadata already dropped py3.8 (needs the pinned ?1.49 from NVIDIA's own index) ? a reminder that any pinned-old-wheel fallback is one yank away from breaking. The primary pick (ffmpeg subprocess) has no such fragility.

Sources: [PyNvVideoCodec docs](https://docs.nvidia.com/video-technologies/pynvvideocodec/read-me/index.html), [PyNvVideoCodec PyPI](https://pypi.org/project/PyNvVideoCodec/), [torchvision GPU decoder PR #5019](https://github.com/pytorch/vision/pull/5019), [torchvision #6161 "Not compiled with GPU decoder support"](https://github.com/pytorch/vision/issues/6161), [torchvision #7745 cuda backend shape/color/seek](https://github.com/pytorch/vision/issues/7745), [torchvision video deprecation (0.22 docs)](https://docs.pytorch.org/vision/0.22/io.html), [DALI installation / enhanced compatibility](https://docs.nvidia.com/deeplearning/dali/user-guide/docs/installation.html), [DALI releases (py3.8 ?1.49)](https://github.com/NVIDIA/DALI/releases), [CUDA forward/enhanced compatibility](https://docs.nvidia.com/deploy/cuda-compatibility/latest/), [decord repo (PyPI CPU-only, build from source for GPU)](https://github.com/dmlc/decord), [decord releases (last v0.6.0)](https://github.com/dmlc/decord/releases), [cudawarped/opencv-python-cuda-wheels](https://github.com/cudawarped/opencv-python-cuda-wheels/releases), [OpenCV cudacodec::VideoReader ref](https://docs.opencv.org/4.x/db/ded/classcv_1_1cudacodec_1_1VideoReader.html), [PyAV h264_cuvid #451](https://github.com/mikeboers/PyAV/issues/451), [PyAV DLPack/CUDA PR #2155](https://github.com/PyAV-Org/PyAV/pull/2155), [NVIDIA FFmpeg transcoding guide (hwdownload behavior)](https://developer.nvidia.com/blog/nvidia-ffmpeg-transcoding-guide/).